UNIVERSITÄT
LEIPZIG

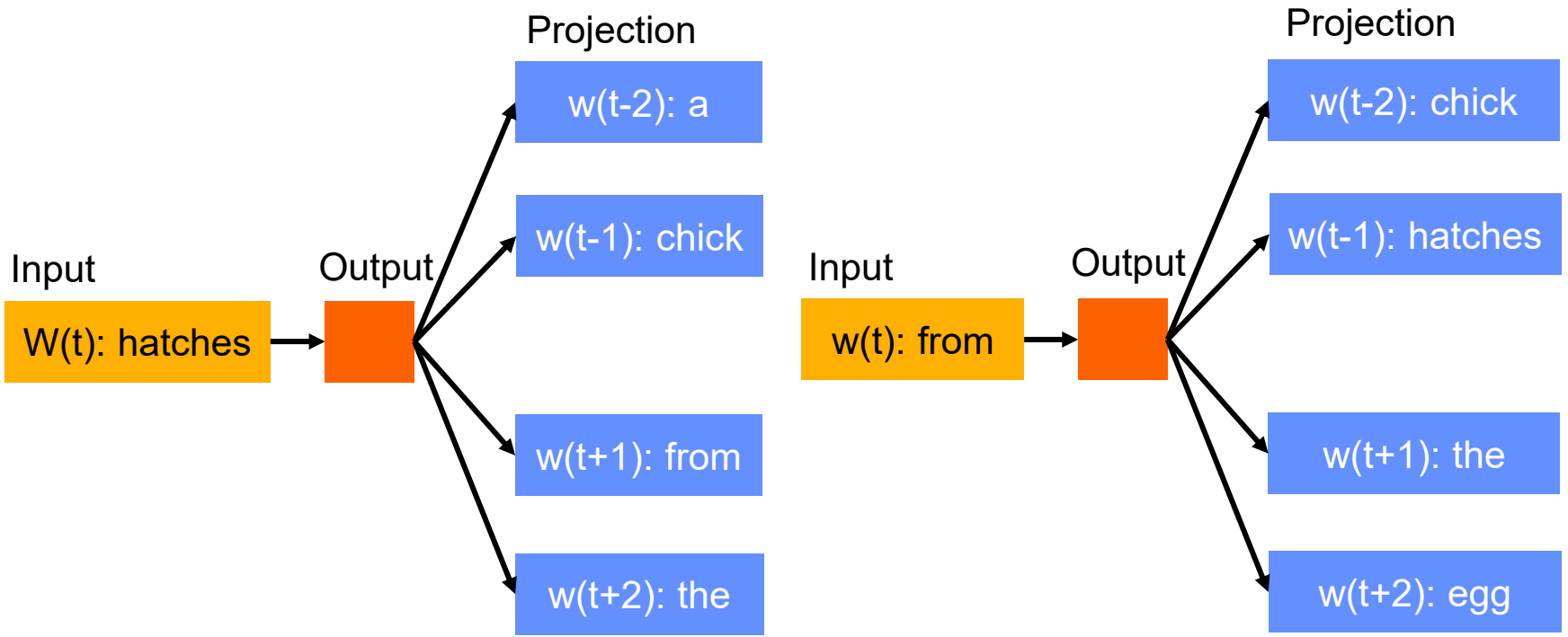
Transformers 1: Neural Networks and Attention

Natural Language Processing

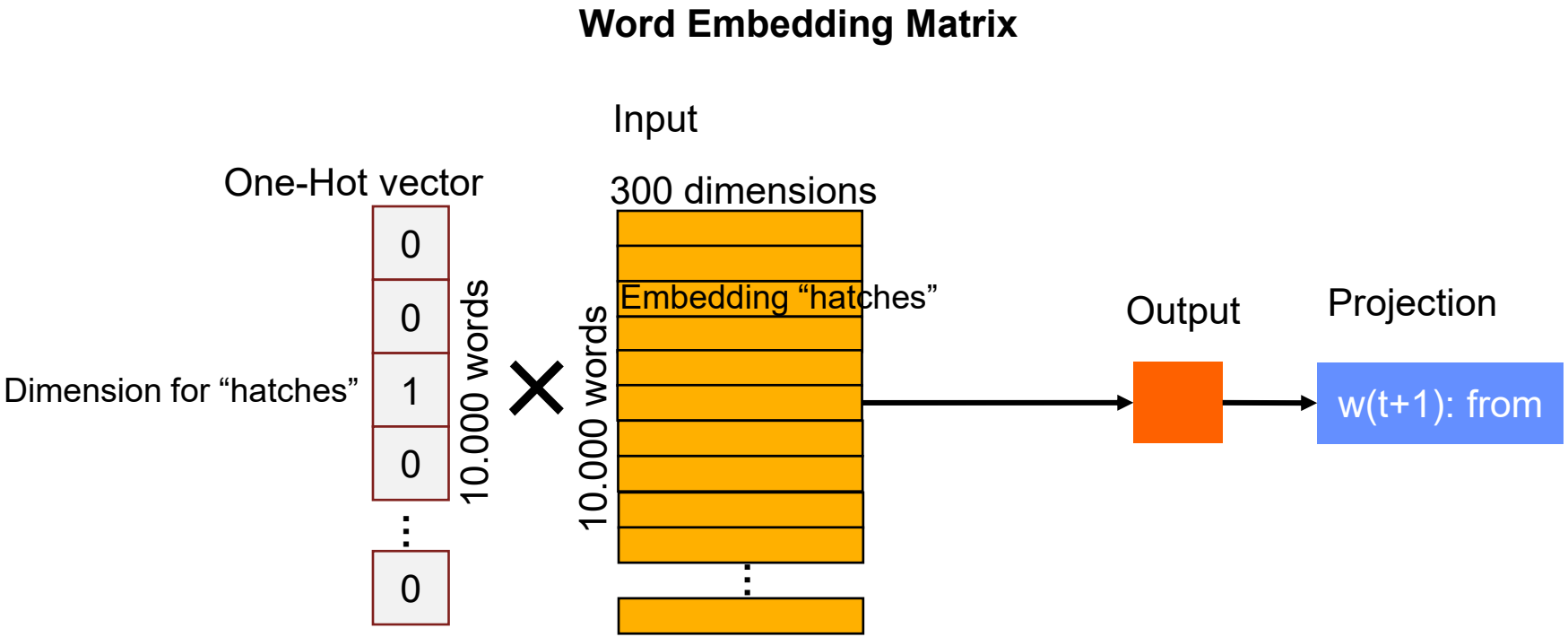
Leonie Weissweiler

RECAP

SKIP-GRAM: A CHICK HATCHES FROM THE EGG

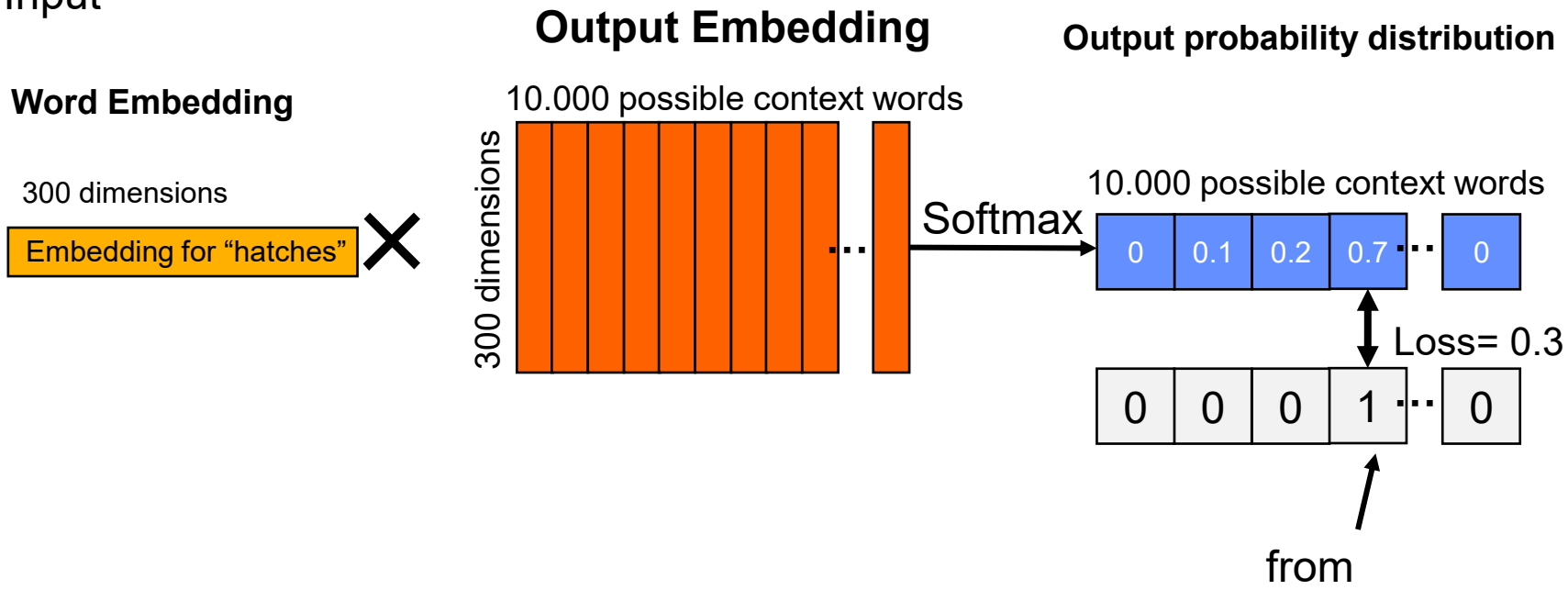


SKIP-GRAM

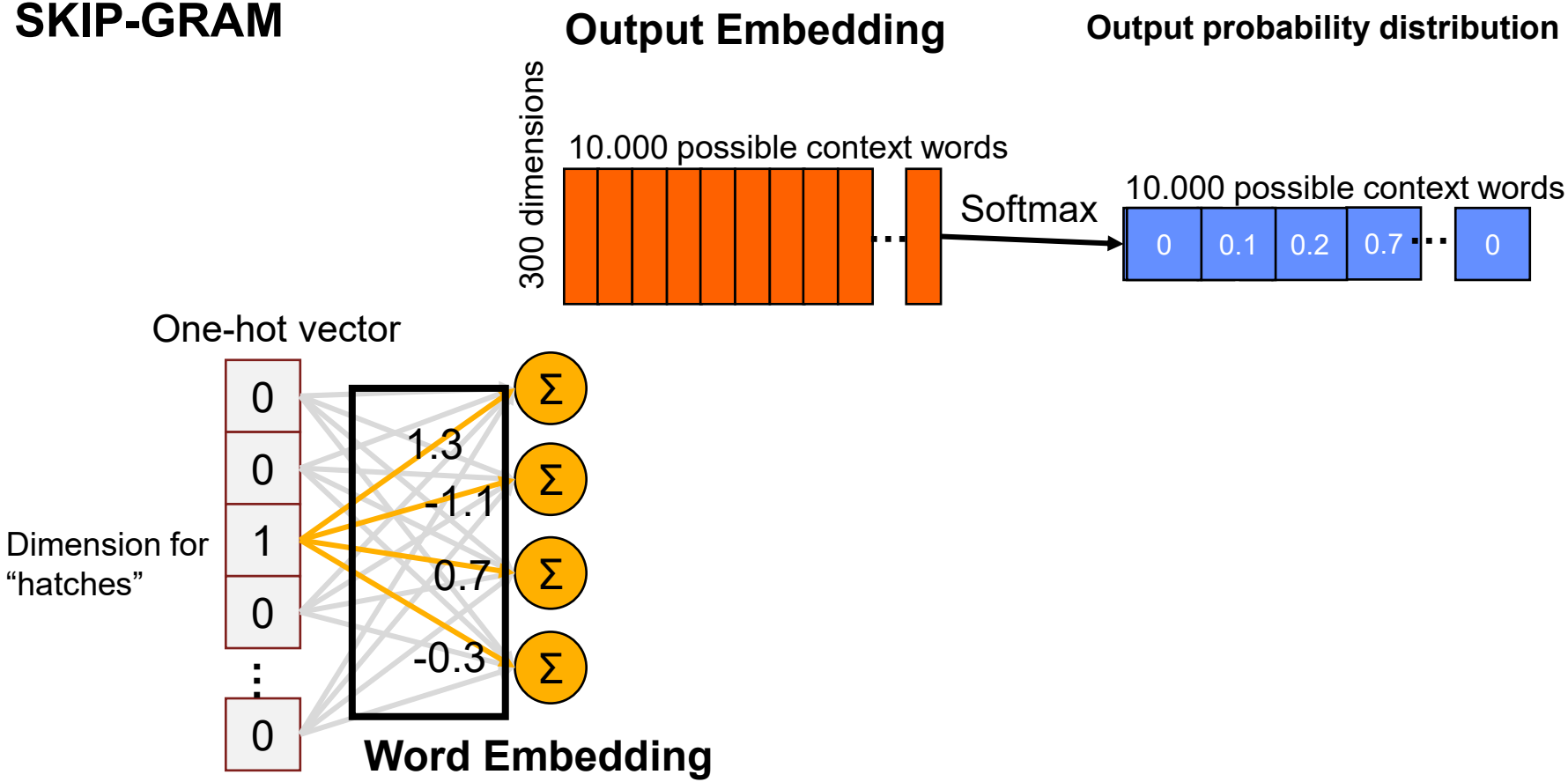

Word Embedding Matrix
Input
One-Hot vector
300 dimensions
0
0
1
0
0
10.000 words
Dimension for “hatches”
10.000 words
Embedding “hatches”
Output
Projection
w(t+1): from

SKIP-GRAM

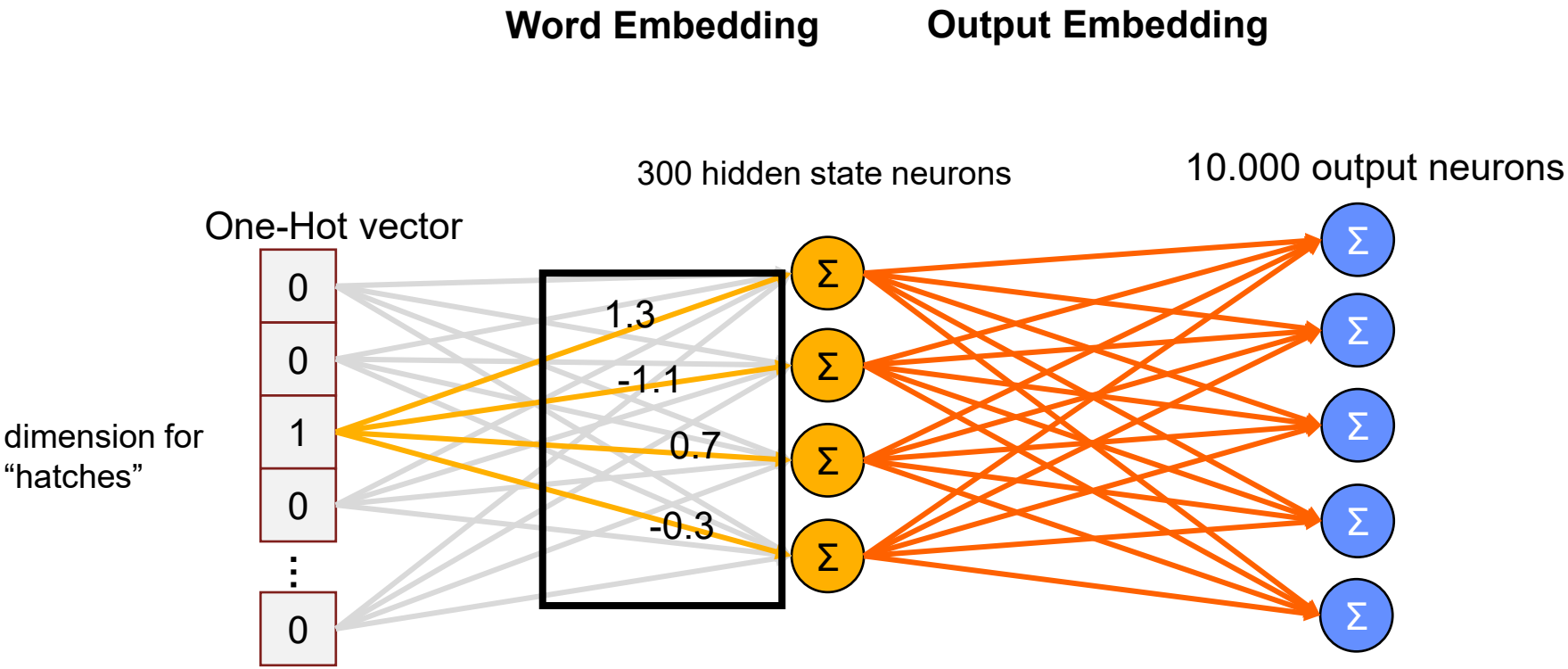
Attention! Not the actual word embedding, just model-internal



SKIP-GRAM


Output Embedding
Output probability distribution
300 dimensions
10.000 possible context words
Softmax
10.000 possible context words
0
0.1
0.2
0.7
0
One-hot vector
Dimension for "hatches"
0
0
1
0
0
1.3
-1.1
0.7
-0.3
Σ
Σ
Σ
Σ
Word Embedding

SKIP-GRAM

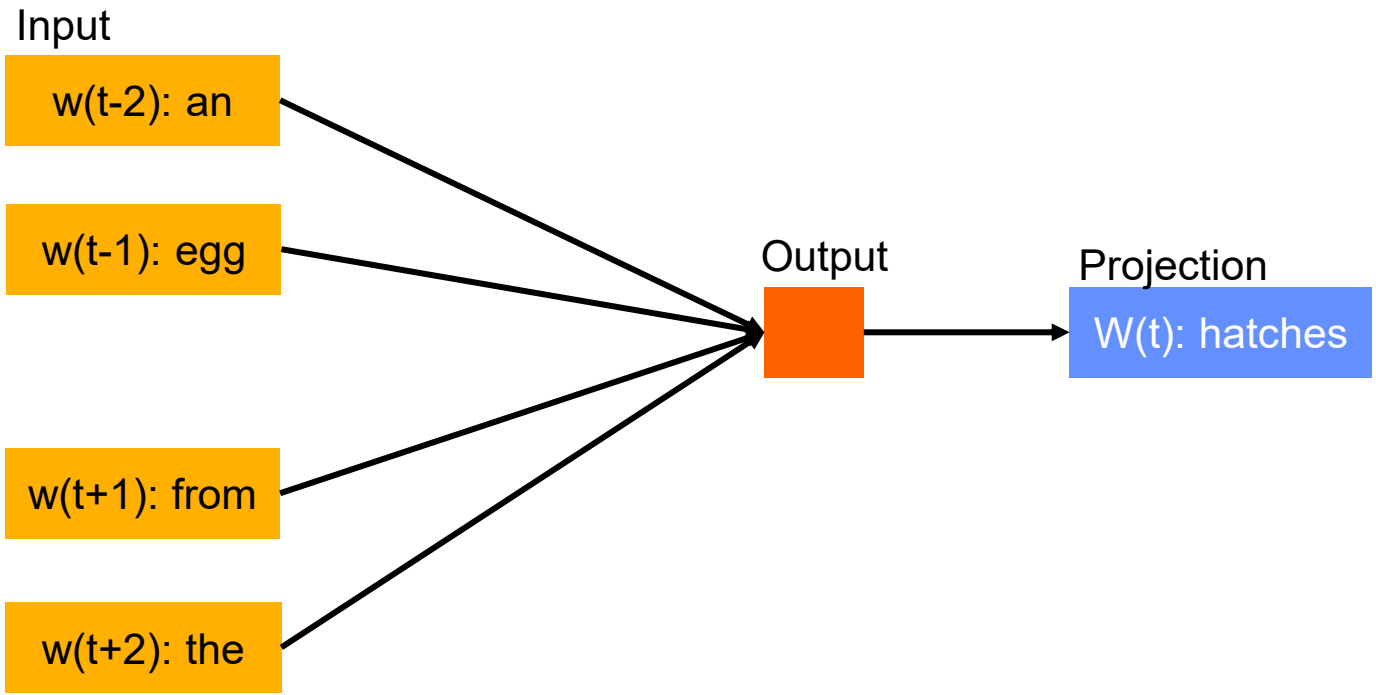

Word Embedding
Output Embedding
300 hidden state neurons
10.000 output neurons
One-Hot vector
dimension for "hatches"
0
0
1
0
0
1.3
-1.1
0.7
-0.3
Σ

THE SKIPGRAM FORMULAS

$$\underbrace{\frac{1}{T}\sum_{t=1}^{T}}_{\text{Sum over all time steps (positions in the sentence)}} \underbrace{\sum_{-c\leq j\leq c, j\neq 0}}_{\text{Sum over all context words(c=2)}} \log \underbrace{p(w_{t+j}|w_t)}_{\text{Probability of the context word given the focus word}}$$

$$p(w_O|w_I) = \frac{\exp\left(\overbrace{v'_{w_O}{}^{\top} v_{w_I}}^{\text{Scalar product of word embedding and context vector}}\right)}{\sum_{w=1}^{W}\exp\left(\underbrace{v'_w}_{\text{word embedding}}{}^{\top}\underbrace{v_{w_I}}_{\text{context vector}}\right)} \Bigg\} \text{softmax}$$

CONTINUOUS BAG OF WORDS (CBOW)



EXAMPLE: D = { “A HORSE, A HORSE, MY KINGDOM FOR A HORSE!” }

R:

k	(t_L, t_R)
1:	(o, r)
2:	(␣h, or)
3:	(␣hor, s)
	⋮

$V = \{$,, !, ␣a, ␣f, ␣h, ␣k, ␣m, A, d, e, f, g, h, i, k, m, n, o, r, s, y, dom, ␣for, ␣hor, ␣hors, ␣horse, ␣king, ␣my, or, ...$\}$

$I = \{$ (A; 1), (␣a; 2), (␣for; 1), (␣horse; 3), (␣king, dom; 1), (␣my; 1), (,; 2), (!; 1) $\}$

d': ((A), (␣h, o, r, s, e), (,), (␣a), (␣h, o, r, s, e), (,), (␣m, y), (␣k, i, n, g, d, o, m), (␣f, o, r), (␣a), (␣h, o, r, s, e), (!))

1: ((A), (␣h, or, s, e), (,), (␣a), (␣h, or, s, e), (,), (␣m, y), (␣k, i, n, g, d, o, m), (␣f, or), (␣a), (␣h, or, s, e), (!))

2: ((A), (␣hor, s, e), (,), (␣a), (␣hor, s, e), (,), (␣m, y), (␣k, i, n, g, d, o, m), (␣f, or), (␣a), (␣hor, s, e), (!))

3: ((A), (␣hors, e), (,), (␣a), (␣hors, e), (,), (␣m, y), (␣k, i, n, g, d, o, m), (␣f, or), (␣a), (␣hors, e), (!))

⋮

k: ((A), (␣horse), (,), (␣a), (␣horse), (,), (␣my), (␣king), (dom), (␣for), (␣a), (␣horse), (!))

QUESTIONS?

LINEAR CLASSIFIERS

NAMED ENTITY RECOGNITION

the seminar at <time> 4 pm will

Condition	Additional Knowledge				Action
Word	Lemma	LexCat	case	SemCat	Tag
the	the	Art	low		
seminar	Seminar	Noun	low		
at	at	Prep	low		stime
4	4	Digit	low		
pm	pm	Other	low	timeid	
will	will	Verb	low		

At each time step, we will predict if a time entity is starting here

NAMED ENTITY RECOGNITION

the seminar at <time> 4 pm will

Condition	Additional Knowledge				Action
Word	Lemma	LexCat	case	SemCat	Tag
the	the	Art	low		
seminar	Seminar	Noun	low		
at	at	Prep	low		stime
4	4	Digit	low		
pm	pm	Other	low	timeid	
will	will	Verb	low		

- Our features represent this table using binary variables
- E.g. in the lemma column
  - Most features will be false (=off=0)
  - The lemma features that will be true (=on=1) are:
    - -3_lemma_the
    - -2_lemma_Seminar
    - -1_lemma_at
    - +1_lemma_4
    - +2_lemma_pm
    - +3_lemma_will

CLASSIFICATION

- To classify, we will take the dot product of the feature vector with a learned weight vector
- We will say that the class is true (i.e., we should insert a <stime> here) if the dot product is >0, and false otherwise
- Because we might want to shift the decision boundary, we add a feature that is always true
  - This is called the bias
  - By weighting the bias, we can shift where we make the decision

FEATURE VECTOR

We might use a feature vector like this (this example is simplified, really we'd have all features for all positions):

1	Bias term
0	…(e.g. -3_lemma_giraffe)
1	-3_lemma_the
0	…
1	-2_lemma_Seminar
0	…
0	…
1	-1_lemma_at
1	+1_lemma_4
0	…
1	+1_Digit
1	+2_timeid

WEIGHT VECTOR

- Now we'd like the dot product to be >0 if we should insert a <stime> tag
- To encode an intuition that we have, we might for example want three features with a positive weight
  - -1_lemma_at
  - +1_Digit
  - +2_timeid
- We can give them weights of 1 → sum will be 3
- To make sure that we only classify if all three weights are on, let's set the weight on the bias term to -2

DOT PRODUCT

To compute the dot product first take the product of each row, and then sum these

1	Bias term	-2	1*-2	1*-2
0		0	0*0	
1	-3_lemma_the	0	0*0	
0		0	0*0	
1	-2_lemma_Seminar	0	1*0	
0		0	0*0	→ 1
0		0	0*0	
1	-1_lemma_at	1	1*1	1*1
1	+1_lemma_4	0	1*0	
0		0	0*0	
1	+1_Digit	1	1*1	1*1
1	+2_timeid	1	1*1	1*1

LEARNING THE WEIGHT VECTOR

- The general learning task is simply to find a good weight vector!
  - This is sometimes also called “training”
- Basic intuition: you can check weight vector candidates to see how well they classify the training data
  - Better weight vectors get more of the training data right
- So we need some way to make (smart) changes to the weight vector
  - The goal is to make better decisions on the training data

FEATURE EXTRACTION

- We run feature extraction to get the feature vectors for each position in the text
- We typically use a text representation to represent true values (which are sparse)
- Often we design feature templates which describe the feature to be extracted and give the name of the feature (i.e., -1_lemma_XXX)

TRAINING VS TESTING

- When training the system, we have gold standard labels
- When testing the system on new data, we have no gold standard
  - We run the same feature extraction first
  - Then we take the dot product with the weight vector to get a classification decision
- Finally, we have to go back to the original text to write the <stime> tags into the correct positions

TRAINING

Training is automatically adjusting the weight vector so as to better fit the training corpus.

Intuition: make small adjustments to get a better score on the training data.

These all fit our example:

$$\begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \quad \begin{bmatrix} -2.01 \\ 0.04 \\ 0.0004 \\ 0 \\ 1.1 \\ 0 \\ 0 \\ 0.9001 \\ 0 \\ 0 \\ 0.89 \\ 0.91 \end{bmatrix} \quad \begin{bmatrix} -1.99 \\ 0.04 \\ 0.002 \\ 0 \\ 1.101 \\ 0 \\ 0 \\ 0.9111 \\ 0 \\ 0 \\ 0.892 \\ 0.91 \end{bmatrix} \quad \begin{bmatrix} -2.01 \\ 0.043 \\ 0.0003 \\ 0 \\ 1.1 \\ 0 \\ 0 \\ 0.9144 \\ 0 \\ 0 \\ 0.93 \\ 1.01 \end{bmatrix}$$

PERCEPTRON UPDATE

One way to do this is using a so-called perceptron

- Read the training examples one at a time
- For each training example, decide how to update the weight vector
- The perceptron update rule says:
  - If a training example is classified correctly
    - Do nothing (because the current weight vector is fine)
  - If a training example is classified incorrectly
    - Adjust the weight of every active feature by a small amount towards the desired decision
    - So that the example will score a bit better the next time it is observed
  - Intuition we hope that by making many small changes…
    - The weights on important features increase consistently to the desired values which work well on the entire training set
    - The changes to unimportant feature weights will be random (sometimes up, sometimes down), and the weights will tend towards zero (meaning: no effect on the classification)

PERCEPTRON UPDATE

Say that we have -2 0 0 0 …0 0 0 0.5 as our weight vector, and see this training example. Clearly we will get it wrong…

1	Bias term	-2	1*-2	-2
0	-3_lemma_the	0		
1		0		
0		0		
1	-2_lemma_Seminar	0		→ -1.5
0		0		
0		0		
1	-1_lemma_at	0		
1	+1_lemma_4	0		
0		0		
1	+1_Digit	0		
1	+2_timeid	0.5	1*0.5	0.5

PERCEPTRON UPDATE

We change the weight vector by adding 0.1 to all active features. The score is now better, but still wrong.

1	Bias term	1*-1.9	-1.9
0			
1	-3_lemma_the	1*0.1	0.1
0			
1	2_lemma_Seminar	1*0.1	0.1
0			
0			
1	-1_lemma_at	1*0.1	0.1
1	+1_lemma_4	1*0.1	0.1
0			
1	+1_Digit	1*0.1	0.1
1	+2_timeid	1*0.6	0.5

→ -0.8

PERCEPTRON UPDATE

After looking at many other examples, irrelevant features (like -3_lemma_the) are pushed back towards zero, and important features have stronger weights. We have learned a good weight vector for this example, no further update is needed.

1	Bias term	-2.1	1*-2	-2.1
0	-3_lemma_the	0		
1		-0.1	1*-0.1	-0.1
0	-2_lemma_Seminar	0		
1		0.1	1*0.1	0.1
0		0		
0		0		
1	-1_lemma_at	0.7	1*0.7	0.7
1	+1_lemma_4	0		
0		0		
1	+1_Digit	1.1	1*1.1	1.1
1	+2_timeid	1.2	1*1.2	1.2

→ 0.9

LINEAR MODELS WITH A TINY BIT OF MATHS

$$h(X) = X \cdot \Theta^T \qquad X = \begin{bmatrix} x_0 = 1 \\ x_1 = 1 \\ x_2 = 0 \\ \dots \\ x_{101} = 1 \\ x_{102} = 0 \\ \dots \\ x_{201} = 1 \\ x_{202} = 0 \\ \dots \\ x_{301} = 1 \\ x_{302} = 0 \\ \dots \end{bmatrix} \qquad \Theta = \begin{bmatrix} w_0 = 1.00 \\ w_1 = 0.01 \\ w_2 = 0.01 \\ \dots \\ x_{101} = 0.01 \\ x_{102} = 0.01 \\ \dots \\ x_{201} = 0.01 \\ x_{202} = 0.01 \\ \dots \\ x_{301} = 0.01 \\ x_{302} = 0.01 \\ \dots \end{bmatrix}$$

$$\begin{aligned} h(X) &= X \cdot \Theta^T \\ &= x_0 w_0 + x_1 w_1 + \dots + x_n w_n \\ &= 1 * 1 + 1 * 0.01 + 0 * 0.01 + \dots + 0 * 0.01 + 1 * 0.01 \end{aligned}$$

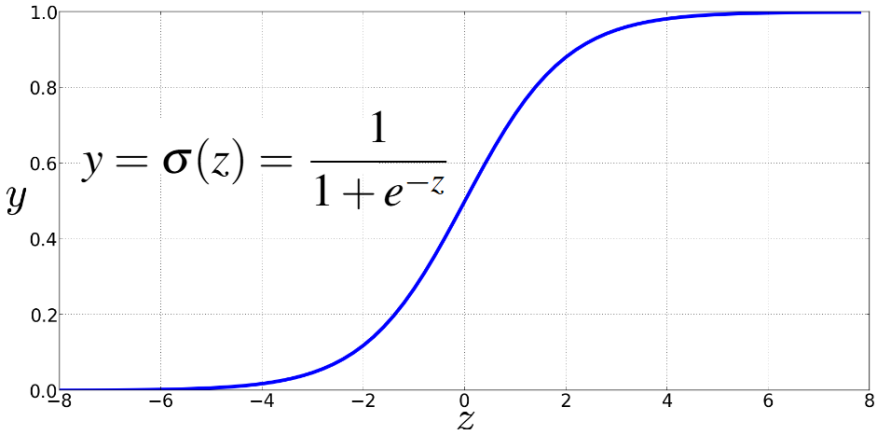
REAL-VALUED FEATURES AND LEARNED VECTORS

$$h(X) = X \cdot \Theta^T \qquad X = \begin{bmatrix} x_0 = 1.0 \\ x_1 = 50.5 \\ x_2 = 52.2 \\ \dots \\ x_{101} = 45.6 \\ x_{102} = 60.9 \\ \dots \\ x_{201} = 40.4 \\ x_{202} = 51.9 \\ \dots \\ x_{301} = 40.5 \\ x_{302} = 35.8 \\ \dots \end{bmatrix} \qquad \Theta = \begin{bmatrix} w_0 = 1.00 \\ w_1 = 0.001 \\ w_2 = 0.02 \\ \dots \\ x_{101} = 0.012 \\ x_{102} = 0.0015 \\ \dots \\ x_{201} = 0.4 \\ x_{202} = 0.005 \\ \dots \\ x_{301} = 0.1 \\ x_{302} = 0.04 \\ \dots \end{bmatrix}$$

$$\begin{aligned} h(X) &= X \cdot \Theta^T \\ &= x_0 w_0 + x_1 w_1 + \dots + x_n w_n \\ &= 1.0 * 1 + 50.5 * 0.001 + \dots + 40.5 * 0.1 + 35.8 * 0.04 \\ &= 540.5 \end{aligned}$$

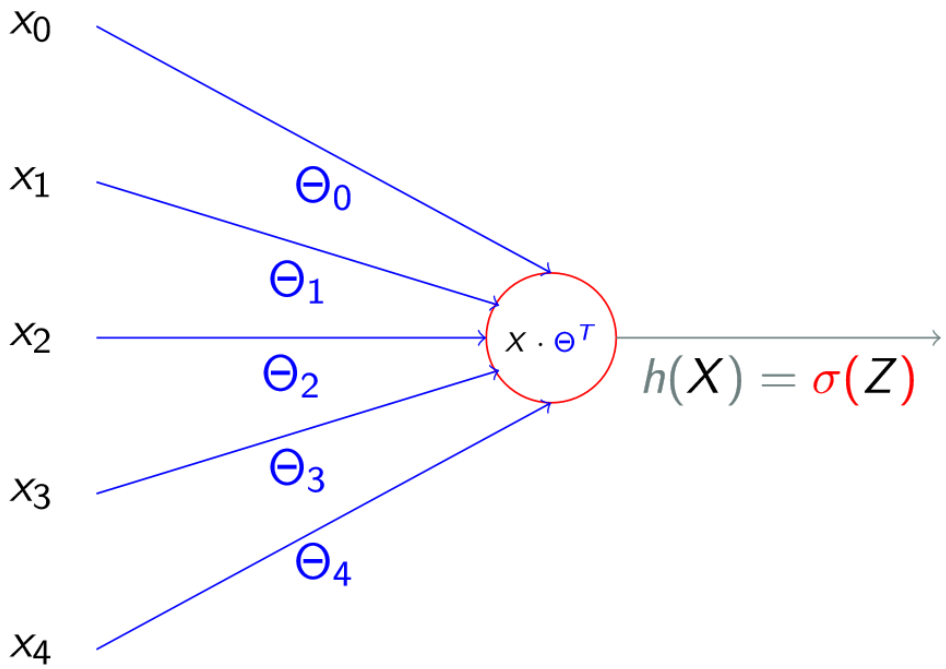
But what does that mean for our decision?

THE SIGMOID FUNCTION



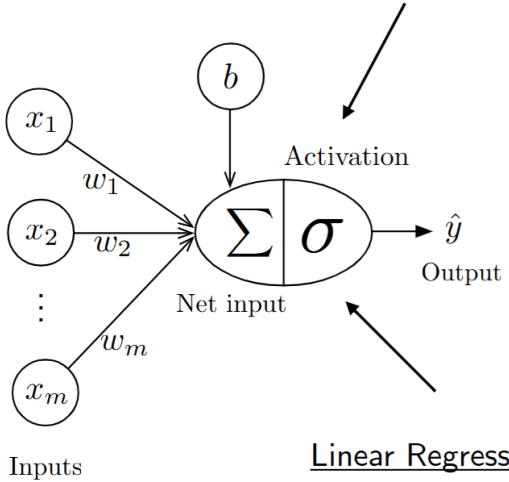
- We'll compute w·x+b
- And then we'll pass it through the sigmoid function: σ(w·x+b)
- And we'll just treat it as a probability

THE PERCEPTRON VISUALLY



- Use a linear model and squash values between 0 and 1 → convert real values to probabilities
- Put threshold at 0.5
- Positive class above threshold, negative class below

LINEAR REGRESSION VS PERCEPTRON



Perceptron: Activation function is the threshold function

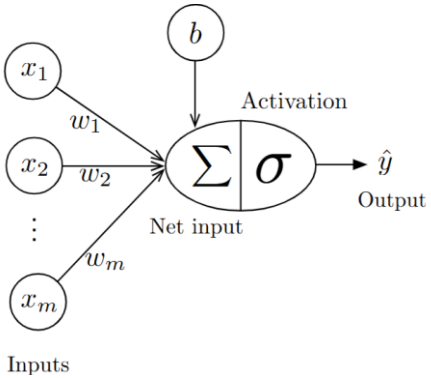
The output is a binary label $\hat{y} \in \{0, 1\}$

Linear Regression: Activation function is the identity function

$$\sigma(x) = x$$

The output is a real number $\hat{y} \in \mathbb{R}$

THE PERCEPTRON UPDATE RULE



$$\sigma\left(\sum_{i=1}^{m} x_i w_i + b\right) = \sigma\left(\mathbf{x}^T\mathbf{w} + b\right) = \hat{y} \qquad \sigma(z) = \begin{cases} 0, & z \leq 0 \\ 1, & z > 0 \end{cases}$$

Let $\mathcal{D} = (\langle \mathbf{x}^{[1]}, y^{[1]} \rangle, \langle \mathbf{x}^{[2]}, y^{[2]} \rangle, ..., \langle \mathbf{x}^{[n]}, y^{[n]} \rangle) \in (\mathbb{R}^m \times \{0,1\})^n$

Initialize $\mathbf{w} := 0^{m-1}$, $\mathbf{b} := 0$

For every training epoch:

For every $\langle \mathbf{x}^{[i]}, y^{[i]} \rangle \in \mathcal{D}$:

(a) $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]T}\mathbf{w} + b)$ ← Compute output (prediction)

(b) $\text{err} := (y^{[i]} - \hat{y}^{[i]})$ ← Calculate error

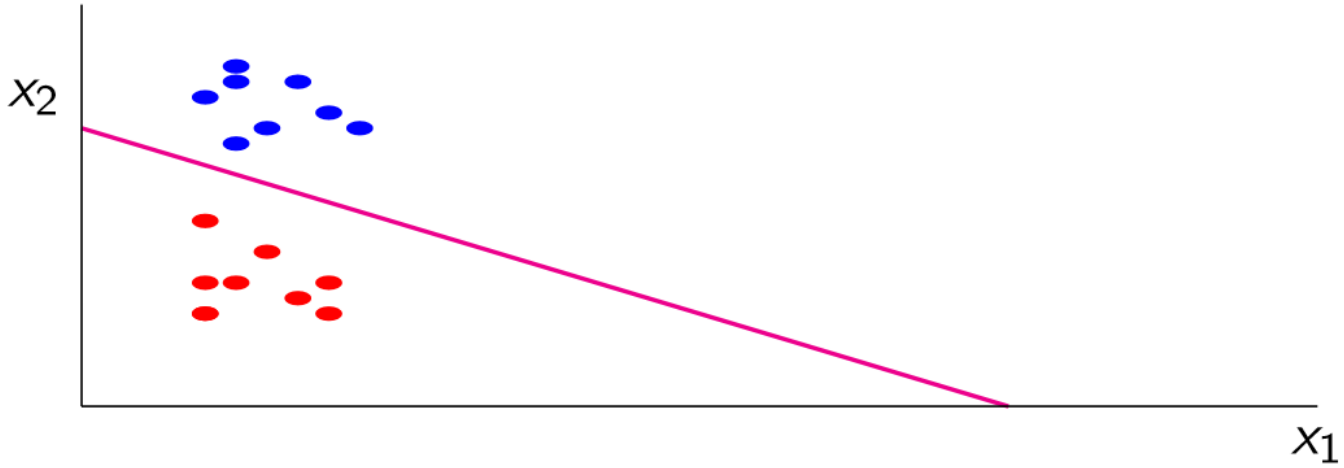
(c) $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]}$, $b := b + err$ ← Update parameters

QUESTIONS?

NEURAL NETWORKS

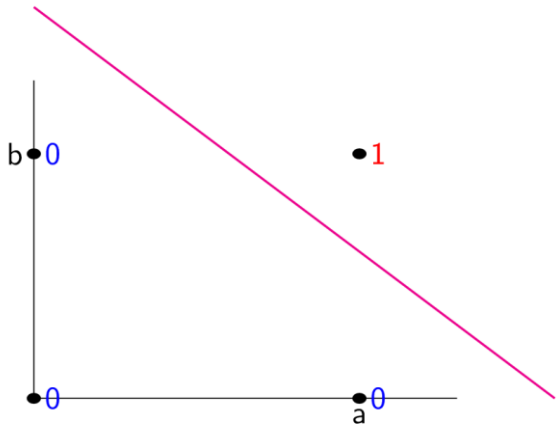
DECISION BOUNDARIES

Linear models define a decision boundary between two classes

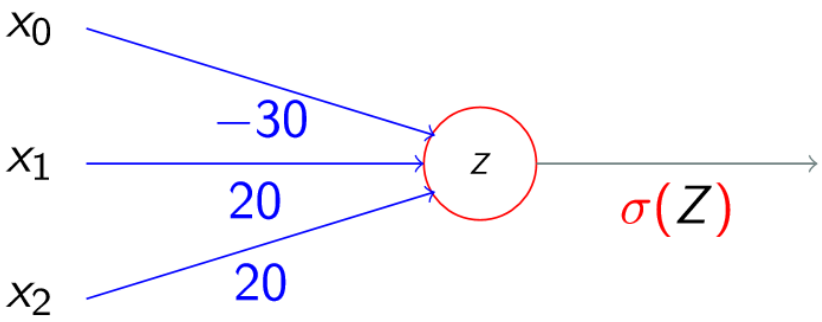


LEARNING A PREDICTOR FOR “AND”

a	b	a ∧ b
0	0	0
0	1	0
1	0	0
1	1	1



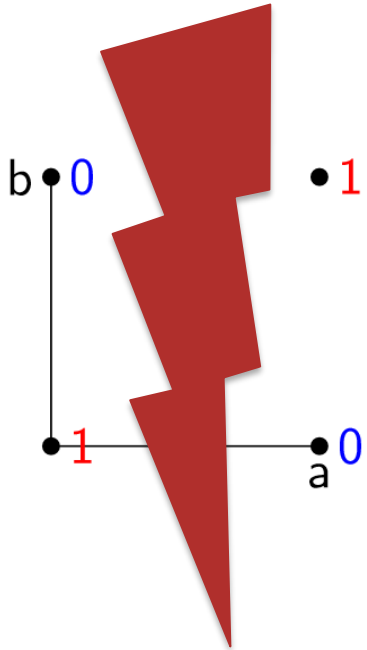
EXAMPLE WEIGHTS FOR “AND”



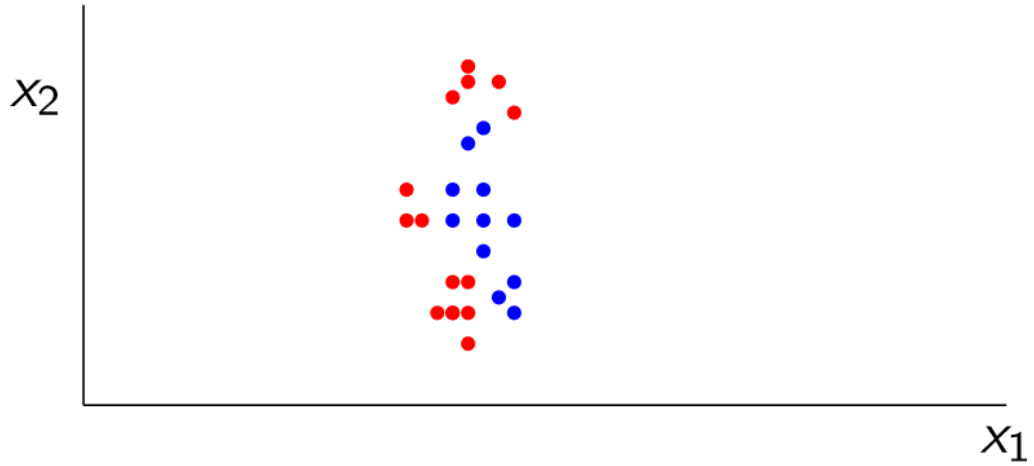
x_0	x_1	x_2	$x_1 \wedge x_2$
1	0	0	$\sigma(1 * -30 + 0 * 20 + 0 * 20) = \sigma(-30) \approx 0$
1	0	1	$\sigma(1 * -30 + 0 * 20 + 1 * 20) = \sigma(-10) \approx 0$
1	1	0	$\sigma(1 * -30 + 1 * 20 + 1 * 20) = \sigma(-10) \approx 0$
1	1	1	$\sigma(1 * -30 + 1 * 20 + 1 * 20) = \sigma(10) \approx 1$

LEARNING A PREDICTOR FOR “XOR”

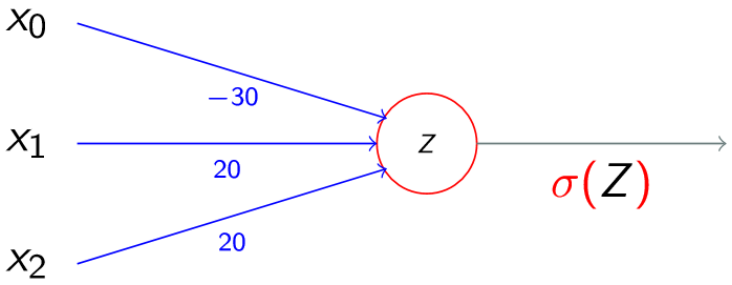
a	b	a *XNOR* b
0	0	1
0	1	0
1	0	0
1	1	1



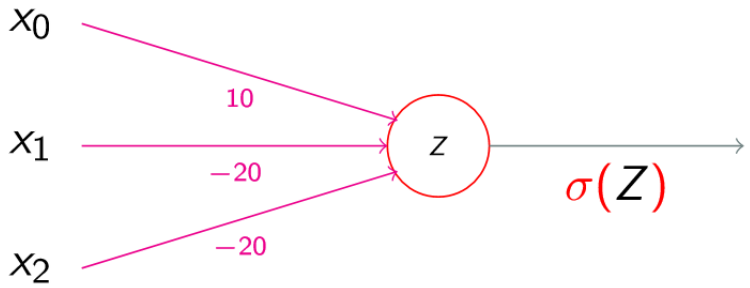
NON-LINEAR DECISION BOUNDARIES



FUNCTION COMPOSITION

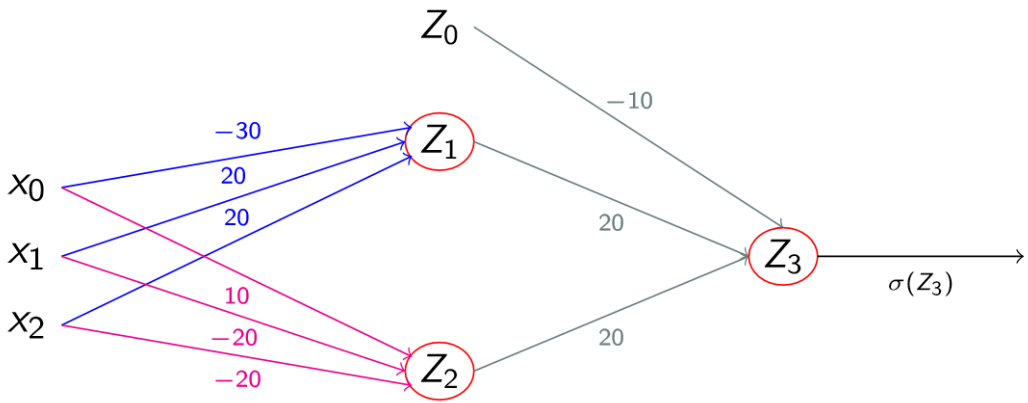


x_0	x_1	x_2	$x_1 \wedge x_2$
1	0	0	≈ 0
1	0	1	≈ 0
1	1	0	≈ 0
1	1	1	≈ 1



x_0	x_1	x_2	$\neg x_1 \wedge \neg x_2$
1	0	0	≈ 1
1	0	1	≈ 0
1	1	0	≈ 0
1	1	1	≈ 0

FUNCTION COMPOSITION



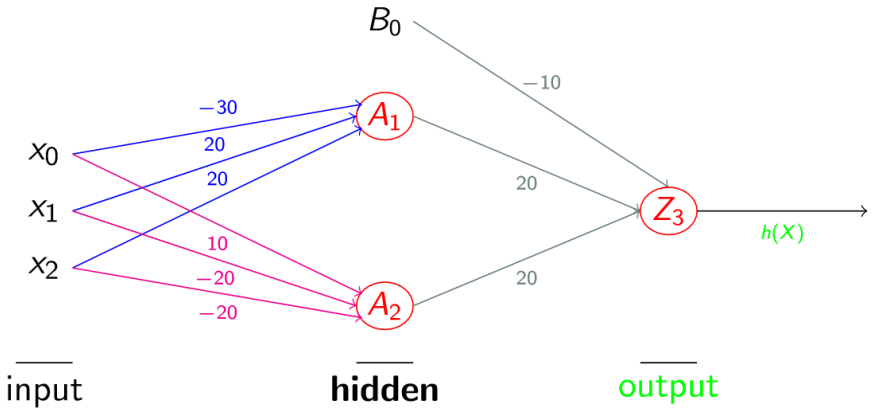
x_0	x_1	x_2	$\sigma(Z_1)$	$\sigma(Z_2)$	$\sigma(Z_3)$
1	0	0	≈ 0	≈ 1	$\sigma(1 * -10 + 0 * 20 + 1 * 20) = \sigma(10) \approx 1$
1	0	1	≈ 0	≈ 0	$\sigma(1 * -10 + 0 * 20 + 0 * 20) = \sigma(-10) \approx 0$
1	1	0	≈ 0	≈ 0	$\sigma(1 * -10 + 0 * 20 + 0 * 20) = \sigma(-10) \approx 0$
1	1	1	≈ 1	≈ 0	$\sigma(1 * -10 + 1 * 20 + 1 * 20) = \sigma(30) \approx 1$

FEEDFORWARD NEURAL NETWORK

We have just created a feedforward neural network with

- 1 input layer X (feature vector)
- 2 weight matrices U = (Θ_1, Θ_2) and V = Θ_3
- 1 hidden layer H composed of:
  - 3 activations $A_1 = \sigma(Z_1)$ and $A_2 = \sigma(Z_2)$ where
    - Z1 = X * Θ_1
    - Z2 = X * Θ_2
  - 1 output unit $h(X) = \sigma(Z_3)$ where
    - Z3 = H * Θ_3

FEEDFORWARD NEURAL NETWORK



Computation of hidden layer **H**

$$A_1 = \sigma(X \cdot \Theta_1)$$

$$A_2 = \sigma(X \cdot \Theta_2)$$

$$B_0 = 1 \text{ (bias term)}$$

Computation of output unit

$$h(X) = \sigma(\mathbf{H} \cdot \Theta_3)$$

FEEDFORWARD NEURAL NETWORK AS MATRIX MULTIPLICATION

Neural network: $h(X) = \sigma(\mathbf{H} \cdot \Theta_n)$

$$\mathbf{H} = \begin{bmatrix} B_0 = 1 \\ A_1 = \sigma(X \cdot \Theta_1) \\ A_2 = \sigma(X \cdot \Theta_2) \\ \cdots \\ A_j = \sigma(X \cdot \Theta_j) \end{bmatrix}$$

Prediction: If $h(X)$ > 0.5, yes. Otherwise, no.

TRAINING NEURAL NETWORKS

Training: Find weight matrices U = (Θ_1, Θ_2) and V = Θ_3 such that $h(X)$ is the correct answer as many times as possible.

→ Given a set T of training examples t_1, …, t_n with correct labels y_i, find

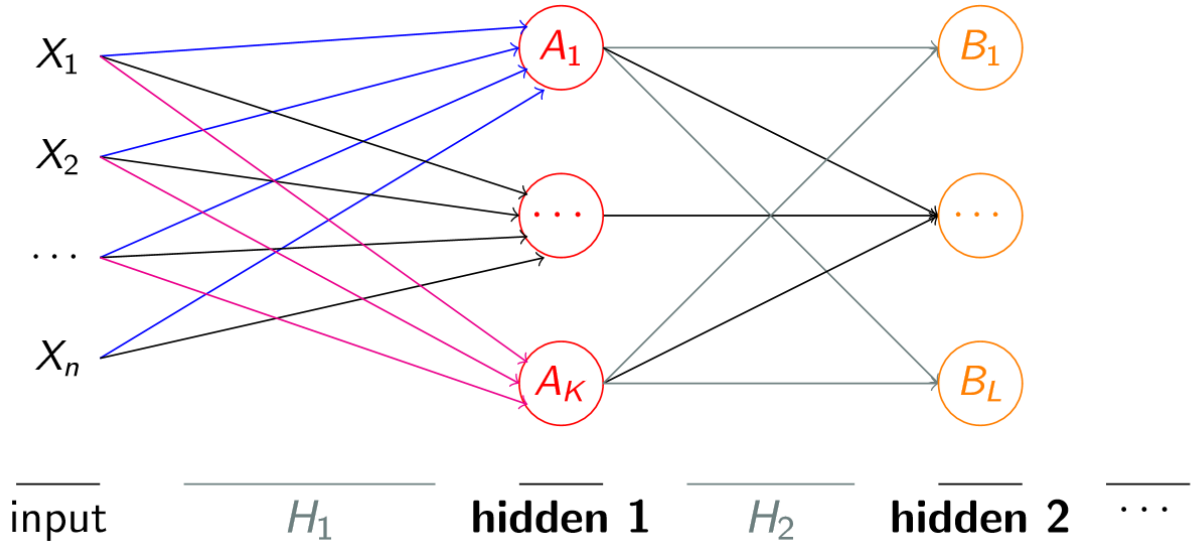
U = (Θ_1, Θ_2) and V = Θ_3 such that $h(X)$ = y_i for as many t_i as possible

→ Computation of $h(X)$ is called forward propagation

→ U = (Θ_1, Θ_2) and V = Θ_3 are adapted with error back propagation

NETWORK ARCHITECTURES

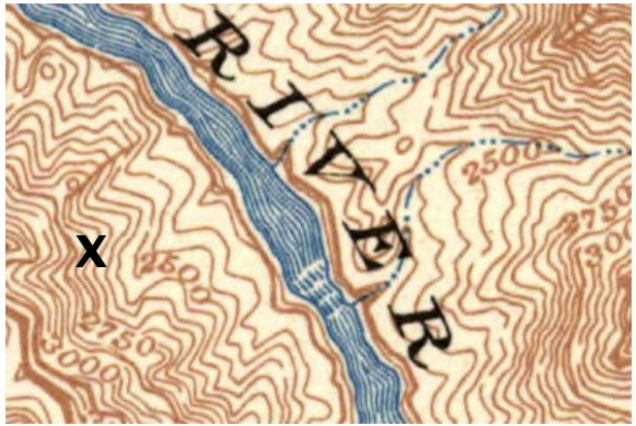
The number of hidden layers and their size can be chosen depending on the complexity of the task and the available resources:



QUESTIONS?

GRADIENT DESCENT

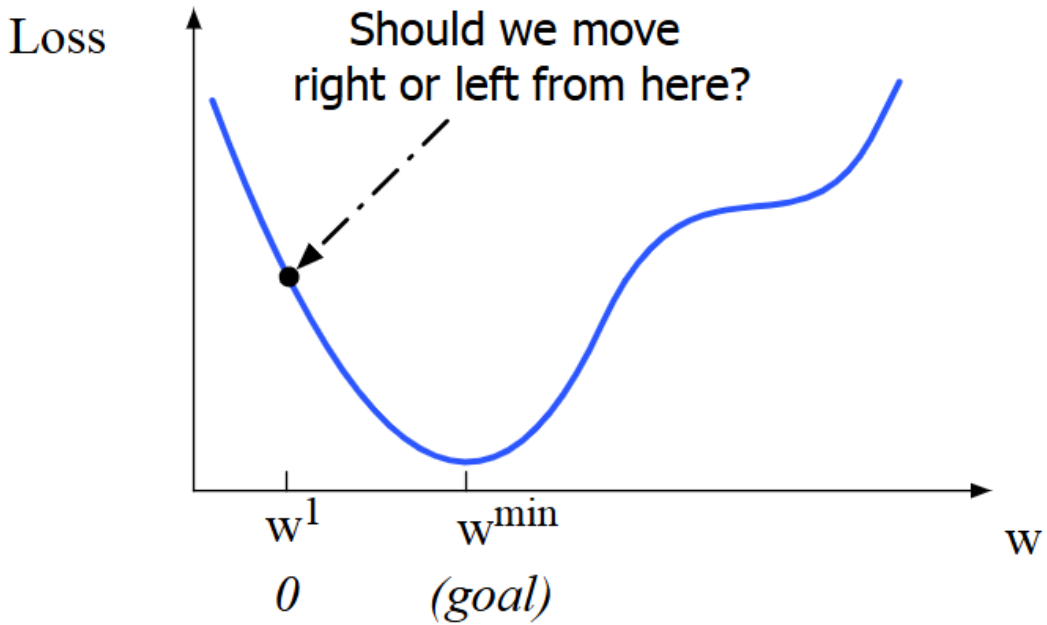
GRADIENT DESCENT INTUITION



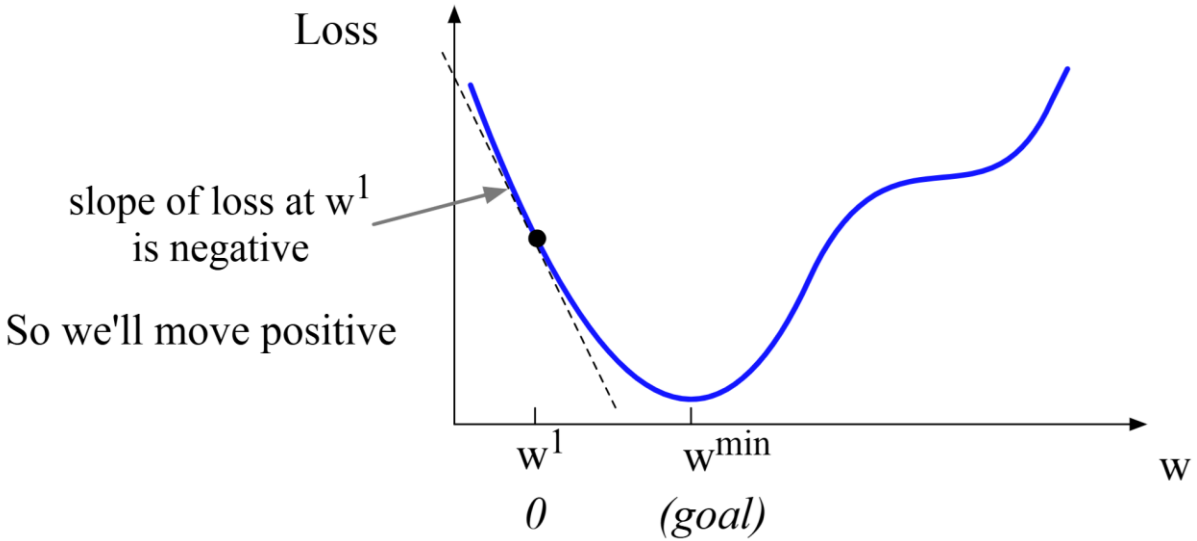
How do I get to the bottom of this river canyon?

- look around me 360◦
- Find the direction of the steepest slope going down
- Go that way

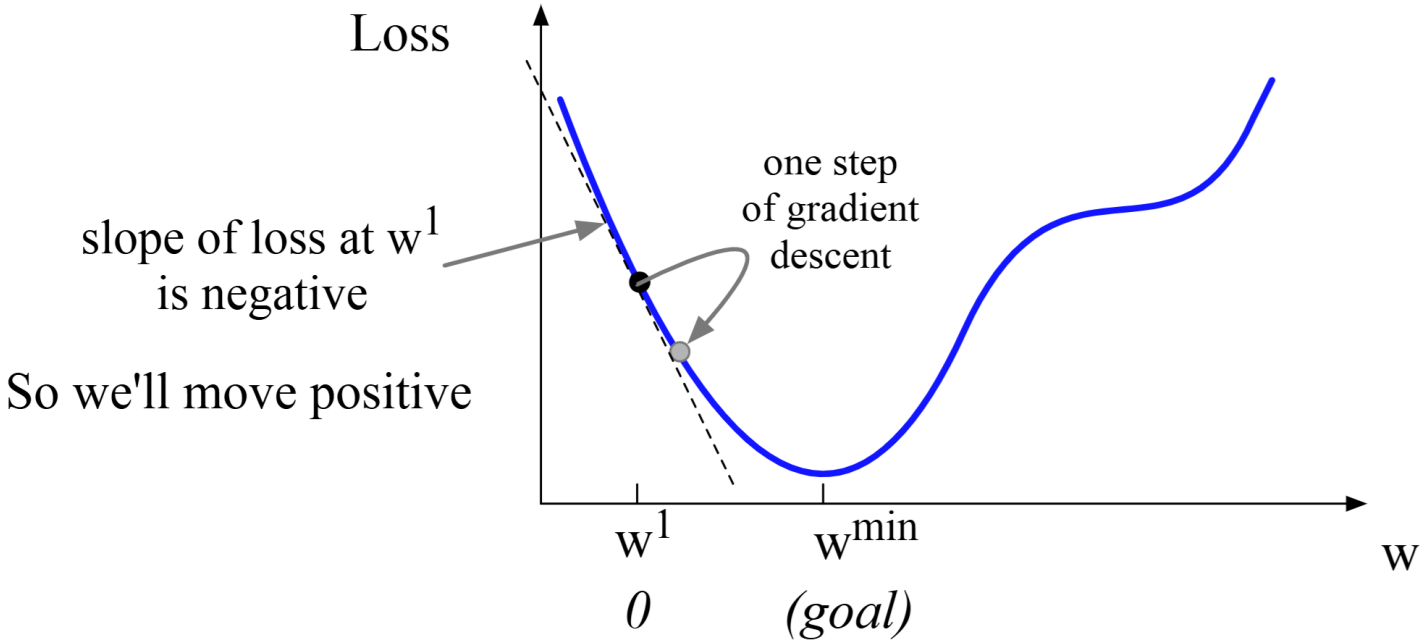
OUR GOAL: MINIMISING THE LOSS



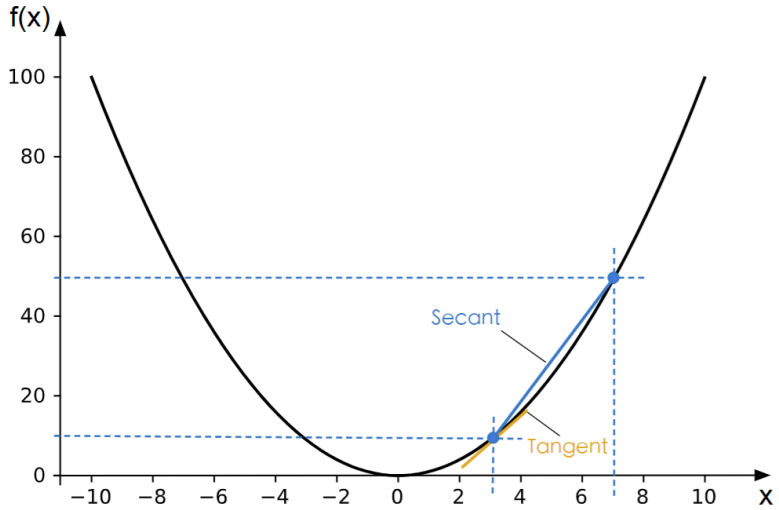
OUR GOAL: MINIMISING THE LOSS



OUR GOAL: MINIMISING THE LOSS



REMINDER: DERIVATIVES

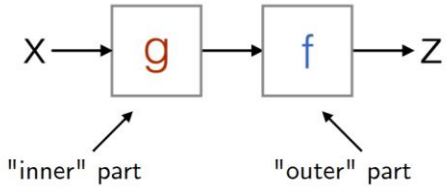


derivative $\frac{d}{dx}x^2 = 2x$

THE CHAIN RULE

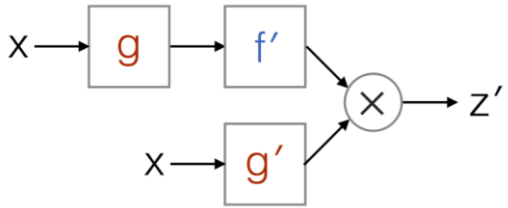
$$F(x) = f(\,g(x)\,) = z$$

Decomposition of some (nested) function:



$$F'(x) = f'(\,g(x)\,)\,g'(x) = z'$$

Derivative of that nested function:



$$\frac{d}{dx}\left[f(g(x))\right] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

SIMPLE EXAMPLE FOR CHAIN RULE

$$\frac{d}{dx}\left[f(g(x))\right] = \frac{df}{dg} \cdot \frac{dg}{dx}$$

$$f(x) = \log(\sqrt{x})$$

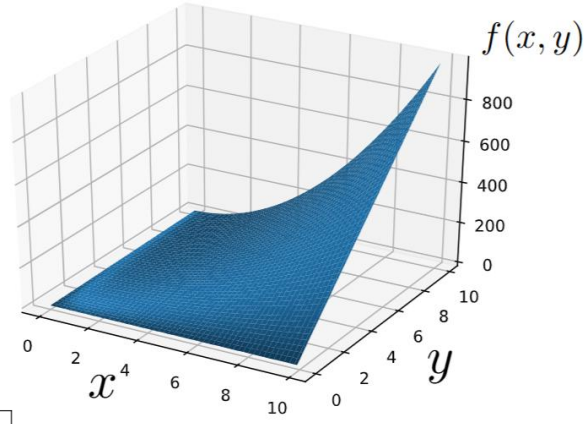
$$\frac{df}{dx} = \frac{d}{dg}\log(g) \cdot \frac{d}{dx}\sqrt{x}$$

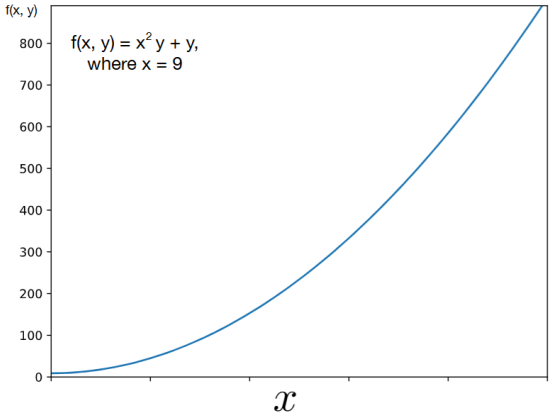
$$\frac{d}{dg}\log(g) = \frac{1}{g} = \frac{1}{\sqrt{x}} \qquad \frac{d}{dx}x^{1/2} = \frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}}$$

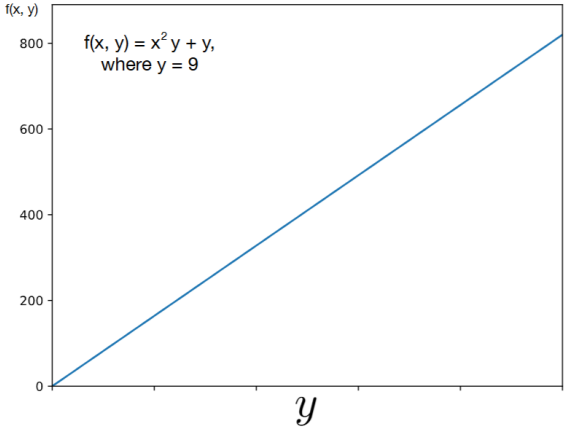
$$\frac{df}{dx} = \frac{1}{\sqrt{x}} \cdot \frac{1}{2\sqrt{x}} = \frac{1}{2x}$$

GRADIENTS OF MULTIPLE VARIABLES AT ONCE

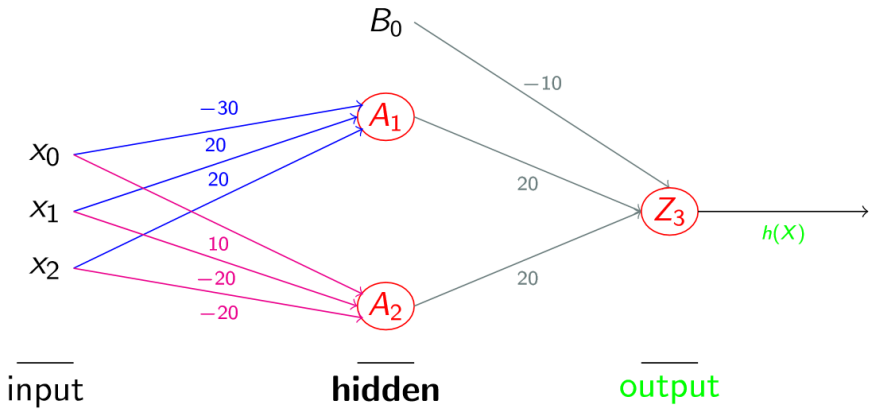
$$f(x, y) = x^2 y + y \qquad \nabla f(x, y) = \begin{bmatrix} 2xy \\ x^2 + 1 \end{bmatrix}$$







BACKPROPAGATION

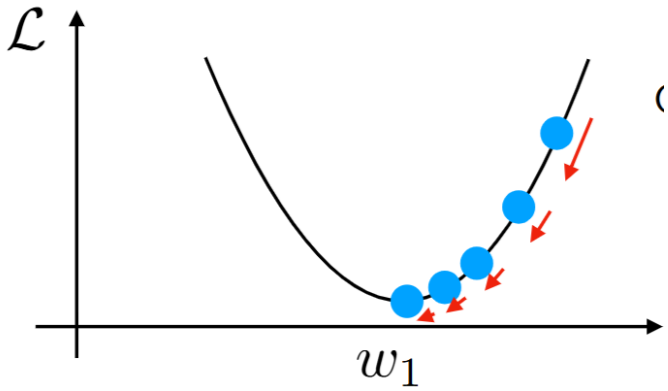


Intuition

- One derivative going in the opposite direction of each arrow
- Here, adjusting A_1 and A_2 are simple derivatives
- Adjusting x_0, x_1 and x_2 requires the chain rule, but we can re-use the derivatives for A_1 and A_2

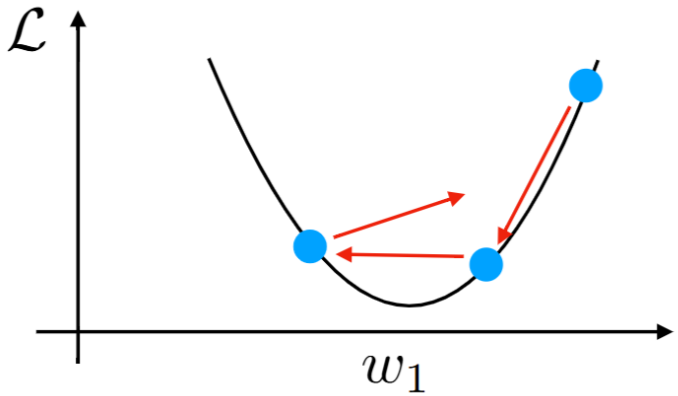
LEARNING RATE

The learning rate and the steepness of the gradient together determine how much we update

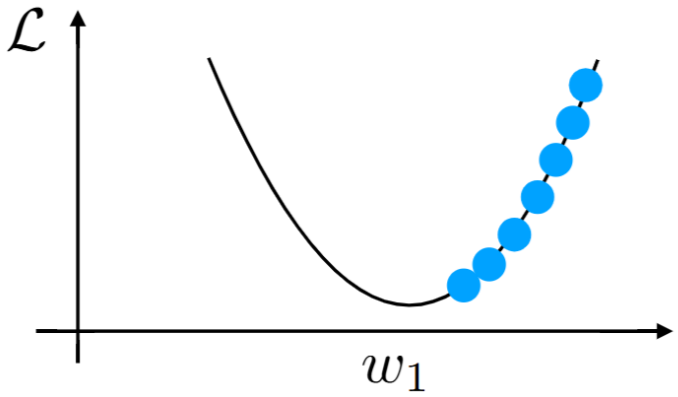


LEARNING RATE

Learning rate too large → overshoot



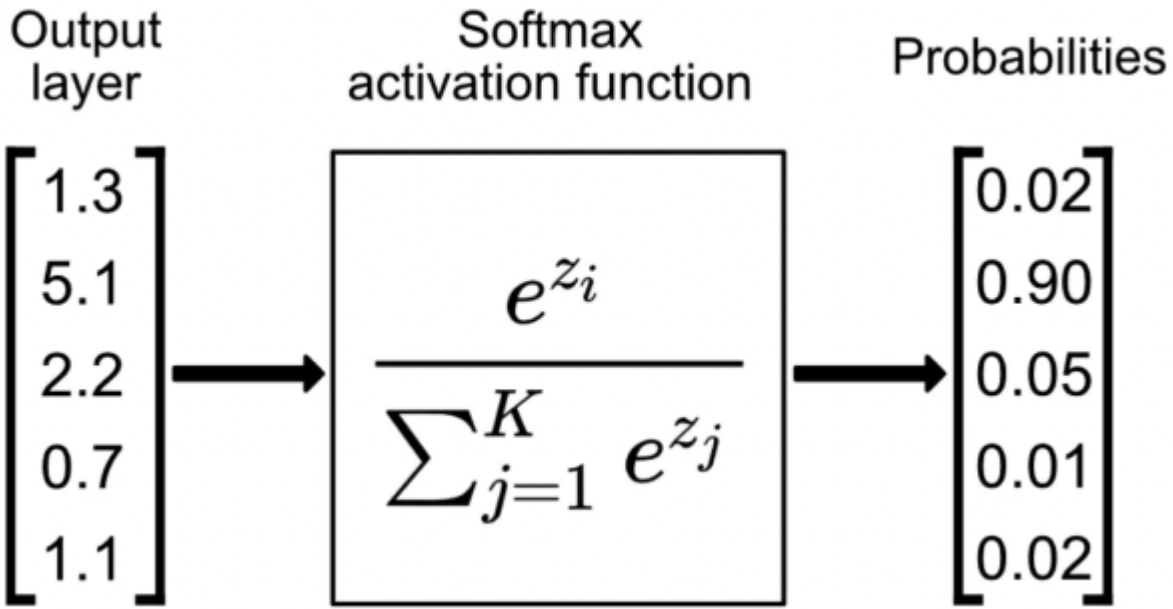
learning rate too small → slow convergence



QUESTIONS?

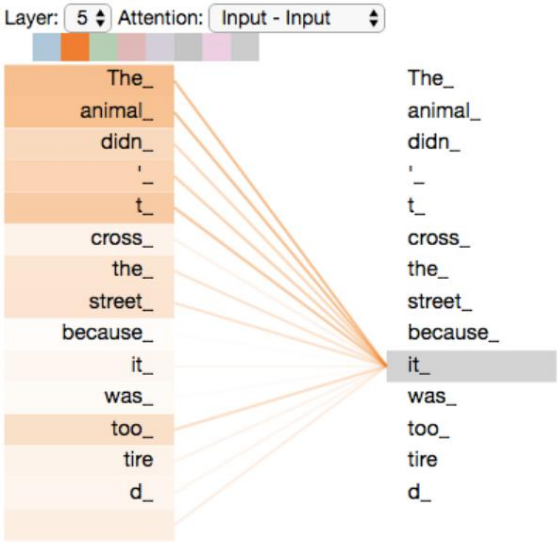
ATTENTION

PREREQUISITE: SOFTMAX



SELF-ATTENTION

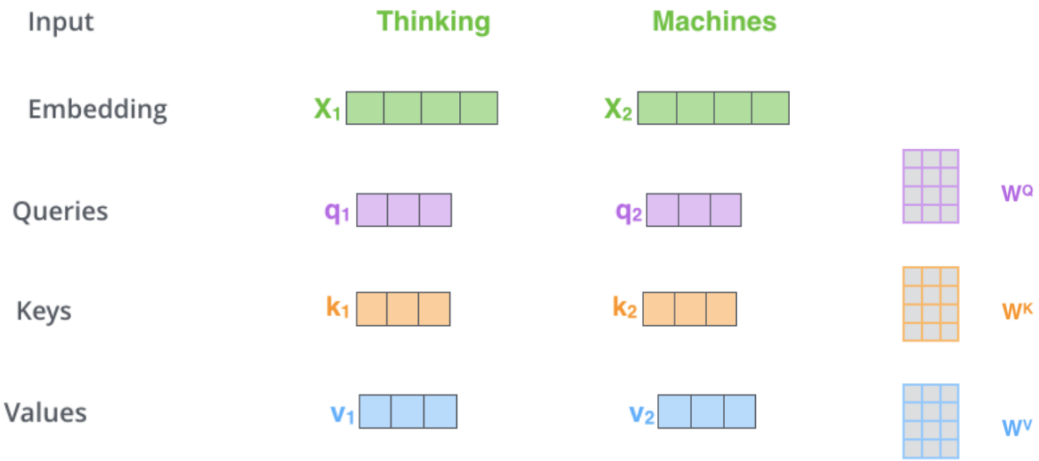
What does “it” in ”The animal didn't cross the street because it was too tired” refer to?



One attention score for each combination of words

How do we get to these scores?

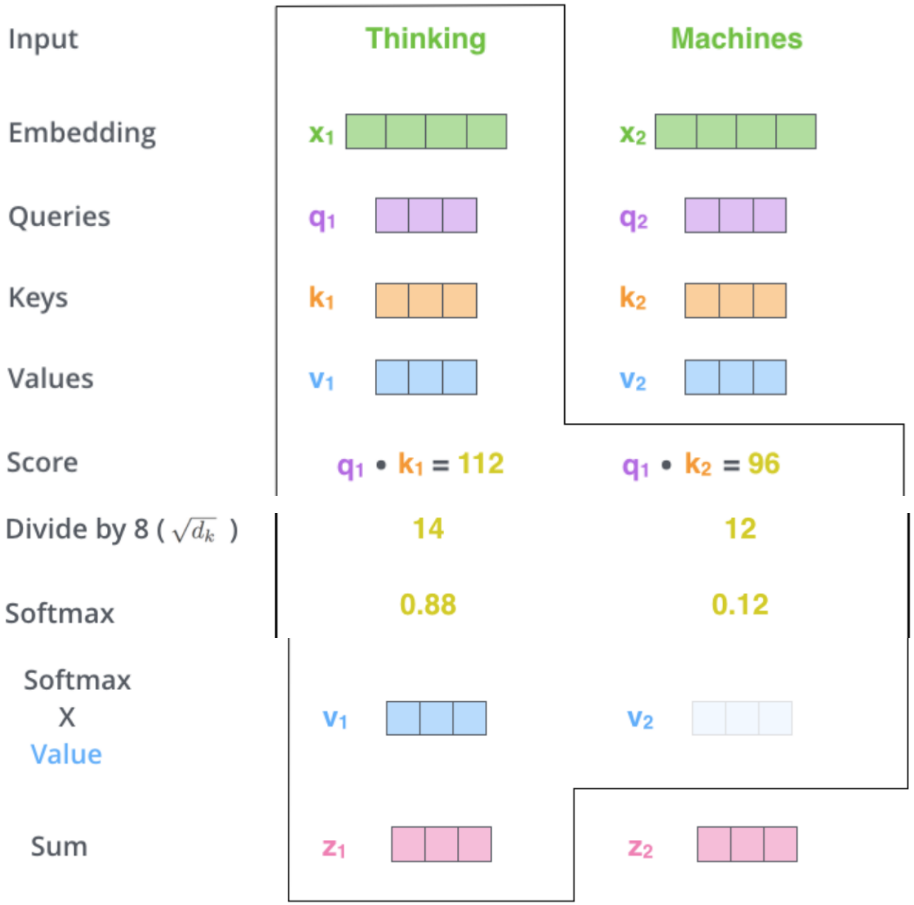
KEY, QUERIES, VALUES



~ what am I looking for?

~ how do I help others determine if they're looking for me?

~ what will I contribute?

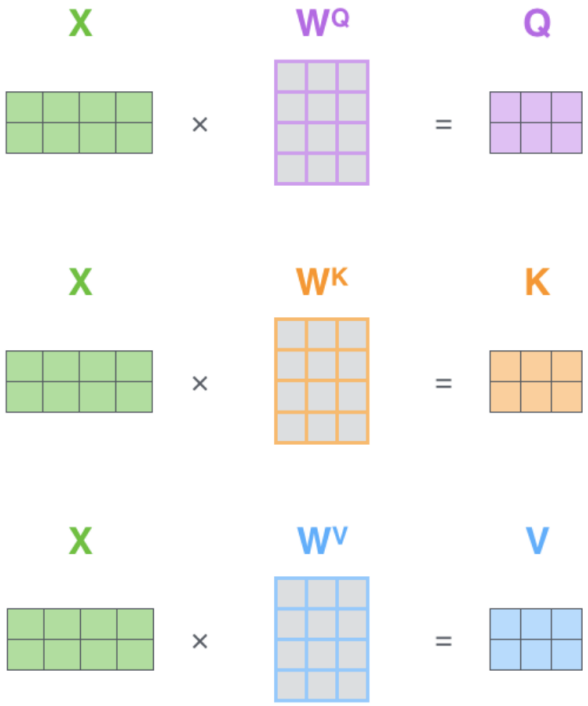


Vector multiplication

Divide by square root of dimensionality
→ more stable gradients

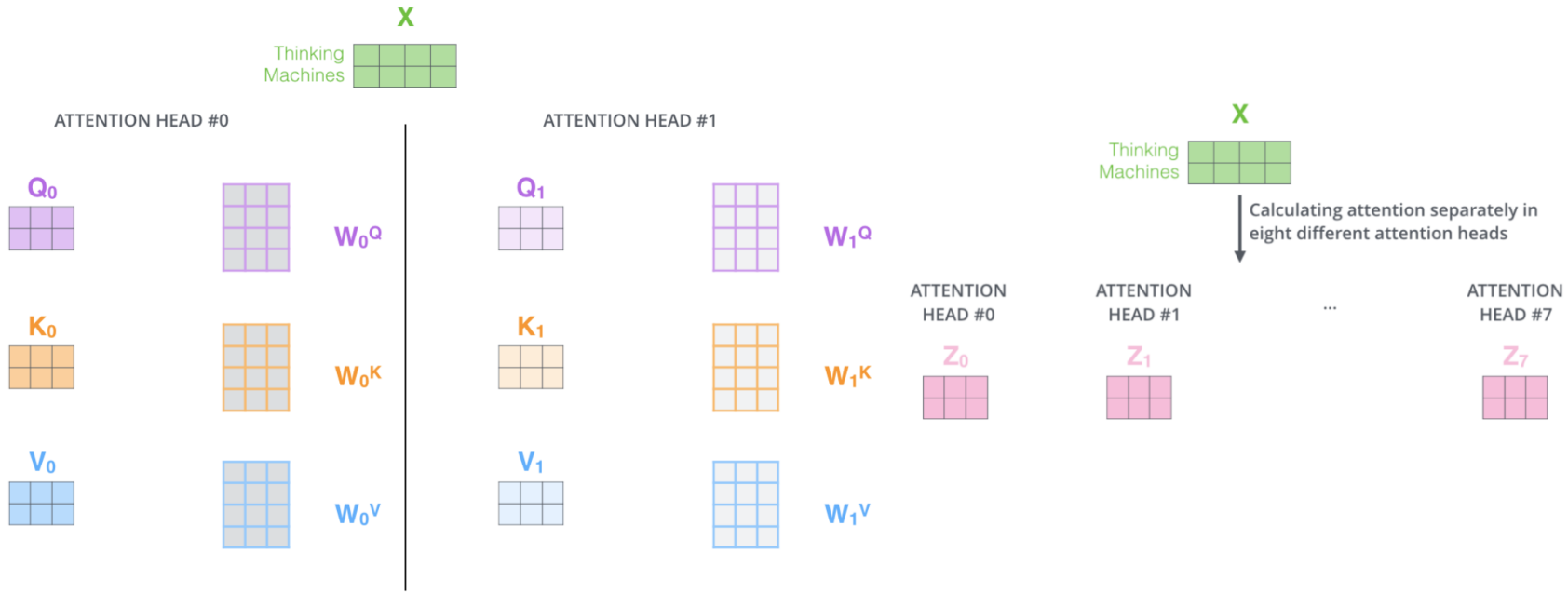
Softmax → drown out irrelevant values of v by multiplying them with tiny numbers

MATRIX CALCULATION OF SELF-ATTENTION



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

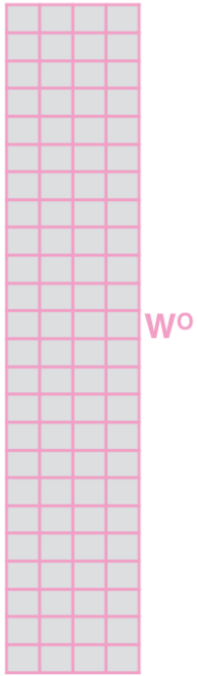
MULTI-HEAD ATTENTION



1) Concatenate all the attention heads

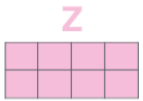
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



1) This is our input sentence*

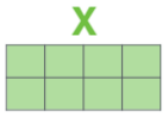
2) We embed each word*

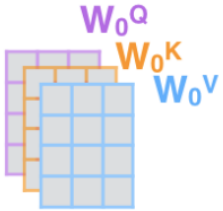
3) Split into 8 heads. We multiply X or R with weight matrices

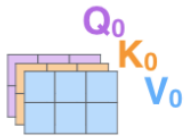
4) Calculate attention using the resulting Q/K/V matrices

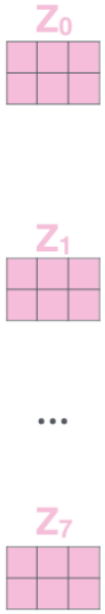
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

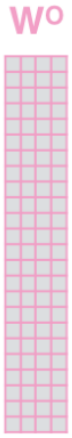
Thinking Machines

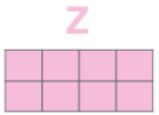




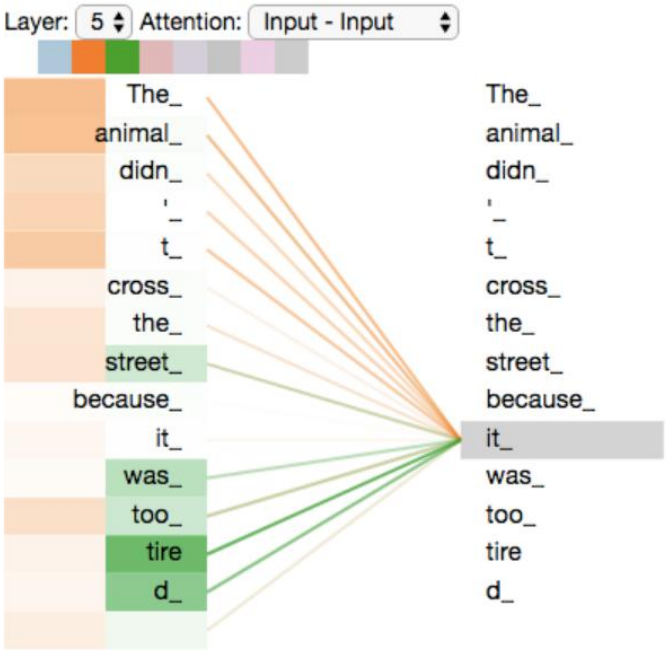


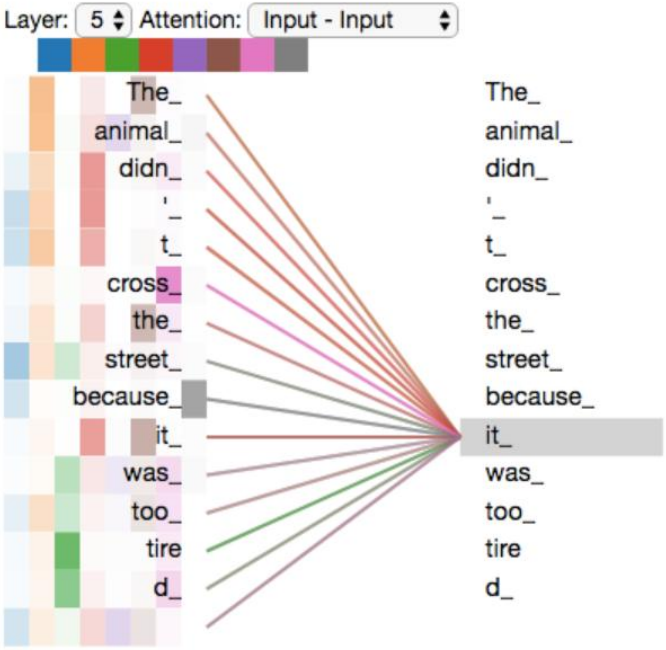






* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one


Layer: 5 Attention: Input - Input
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_


Layer: 5 Attention: Input - Input
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

QUESTIONS?